

PDF to Audiobook

Brief: What It Is and How It Works

Explainer document generated from source

What it does

PDF to Audiobook is a small Python desktop tool: pick one or more PDF files in a window, click Convert, and get back an MP3 per PDF that reads the document's text aloud. It also runs from the command line with no window at all. The whole project is two files, deliberately kept separate: a conversion engine and a GUI shell around it.

Terms used below

GUI A windows-and-buttons interface, built here with **Tkinter**, Python's built-in GUI toolkit.

TTS Text-to-Speech: converting written text to spoken audio. Done here by **gTTS**, a client for Google Translate's public speech-synthesis web service (requires internet, no API key).

PDF text extraction

Reading the words back out of a PDF, done here by **PyPDF2**. PDFs are a visual format, so extraction is not always perfect (scanned/image-only pages yield no text).

Architecture: two modules, one seam

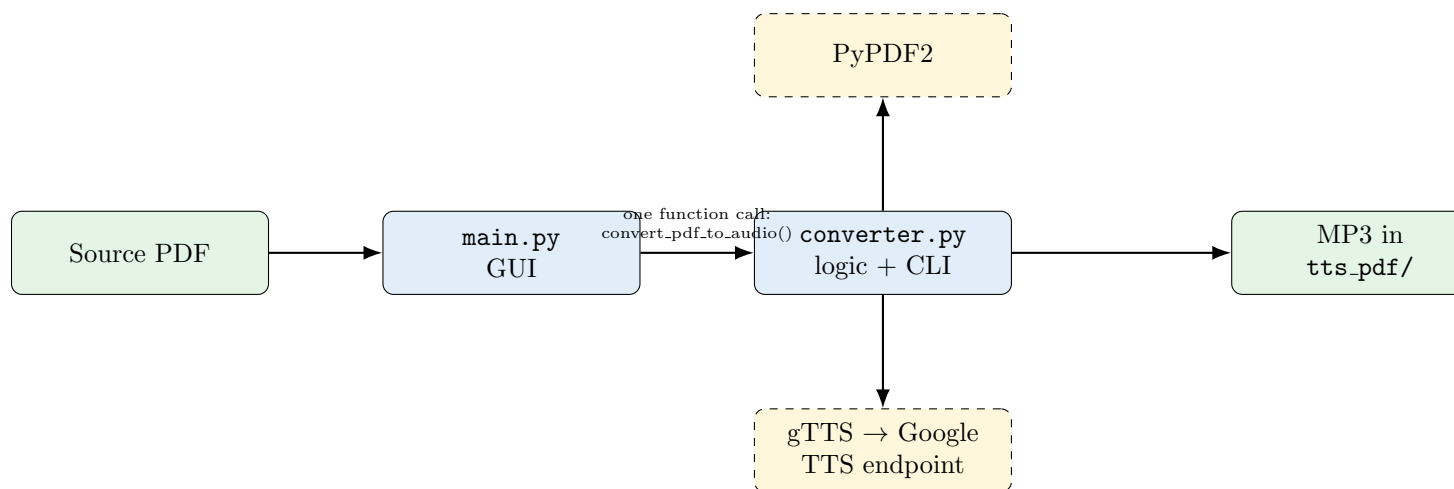


Figure 1: `main.py` never touches `PyPDF2` or `gTTS` directly. It only calls `converter.py`'s public function.

- `converter.py` holds all the real work and has no GUI dependency, so it can run headless (no display) and be tested on its own:
 - `extract_text_from_pdf(pdf_path)` opens the PDF in binary mode, reads every page with PyPDF2, and joins the text with newlines. It substitutes an empty string for any page PyPDF2 fails to extract text from (or `""`), since PyPDF2 returns `None` rather than raising an error on unreadable pages — without this, joining the page list would crash on the first bad page.
 - `convert_pdf_to_audio(pdf_path, output_dir=None, lang="en")` calls the function above, raises a clear `ValueError` if a PDF has no extractable text at all, works out where to save the result using `os.path` functions (defaulting to a `tts_pdf` folder next to the source PDF), synthesizes speech with `gTTS`, and returns the saved MP3 path.
 - A small `argparse`-based CLI at the bottom (guarded by `if __name__ == "__main__":`) lets the same function be run directly: `python3 converter.py file.pdf`. This guard is also what lets `main.py` import the function without triggering the CLI.
- `main.py` builds the Tkinter window (fixed 900×650, three buttons, a status label) and holds one piece of shared state: a global list of chosen filenames.
 - *Select Files / Add Files* open a native PDF-only file picker and either replace or extend that list.
 - *Convert* (`convert_pdf_to_tts`) loops over every selected filename and calls `convert_pdf_to_audio(file` — the single point of contact with the conversion module — wrapping each call in its own `try/except Exception` so one bad PDF does not abort the whole batch. Failures are printed to the console only; the window shows just an aggregate count of how many files converted.

How the pieces talk to each other

The GUI/converter boundary is exactly one function call wide: `main.py` calls `converter.py`'s `convert_pdf_to_audio`, once per selected file, and gets back either a saved path or an exception. There is **no threading**: the conversion loop runs on Tkinter's own main thread, so the window is frozen (cannot be clicked, moved, or resized) for the entire duration of a batch, since each file involves a real network round-trip to Google's TTS service. A single `root.update_idletasks()` call before the loop forces one immediate repaint of the “Converting, please wait...” label, but nothing keeps the UI responsive once the loop is running.

A full conversion, in one picture

Output

Each converted PDF produces one MP3, named `<original_name>.tts.mp3`, saved in a `tts_pdf` folder created next to the source PDF (or a custom folder if one is passed to the CLI's `-o` flag).

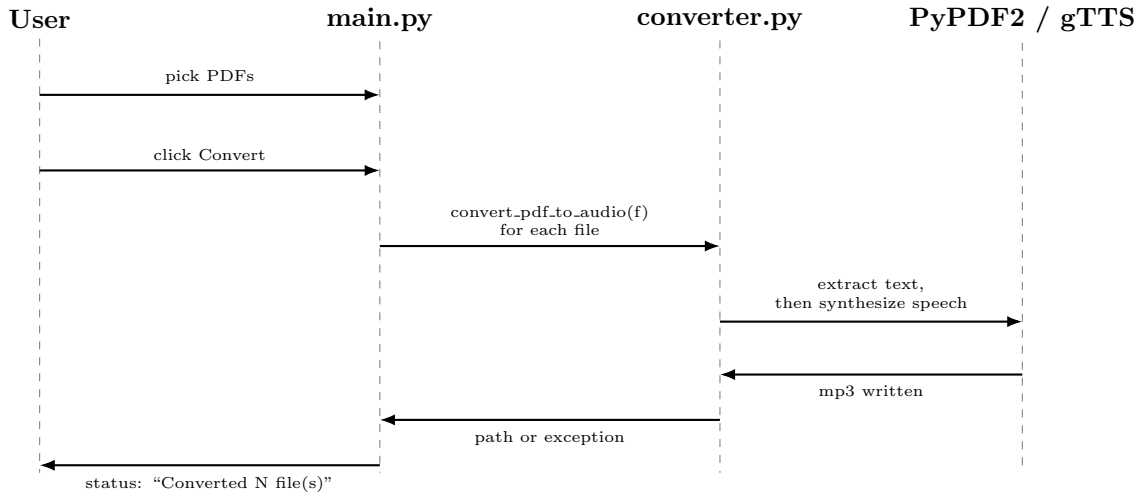


Figure 2: Every file is processed sequentially, on the window’s own thread.

Dependencies

Two pinned third-party packages: `PyPDF2==3.0.1` (text extraction) and `gTTS==2.5.4` (speech synthesis, requires internet, no API key). `Tkinter`, `argparse`, and `os` come from Python’s standard library.

Known limitations (stated in the project itself)

- **No offline fallback.** `gTTS` needs internet access; there is no local TTS engine as a backup.
- **Errors are not surfaced clearly in the GUI.** A network failure and a bad PDF both look the same to the user: part of an aggregate “conversion failed” count, with detail only in the console.
- **The GUI blocks during conversion** because there is no background thread separating the network calls from the window’s event loop.

Summary

The project’s central design decision is a clean split between a GUI-free conversion module built around one function, `convert_pdf_to_audio`, and a thin Tkinter shell that only collects file paths and calls that function once per file. That single-function seam is what makes the conversion logic independently runnable, testable, and reusable from the command line, entirely apart from the window.